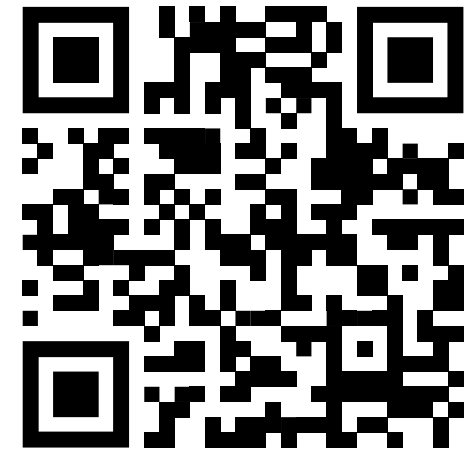


Programmierung in C++

Software-Patterns

Prof. Dr. Bernd Lüdemann-Ravit

Ansprechpartner: Vinzenz Funk



<https://poll.hs-kempten.de/poll/>

Software-Patterns

Einführung



- Dieses Kapitel befasst sich mit dem **Entwurf** von Softwarekomponenten
 - Speziell mit **Entwurfsmustern** (Software-Patterns) und derer Implementierung in C++

Die **Softwarearchitektur** eines Systems beschreibt die Struktur des Systems,

- dessen **Softwarebausteine**,
- **Beziehungen** zwischen diesen und
- deren **Eigenschaften**

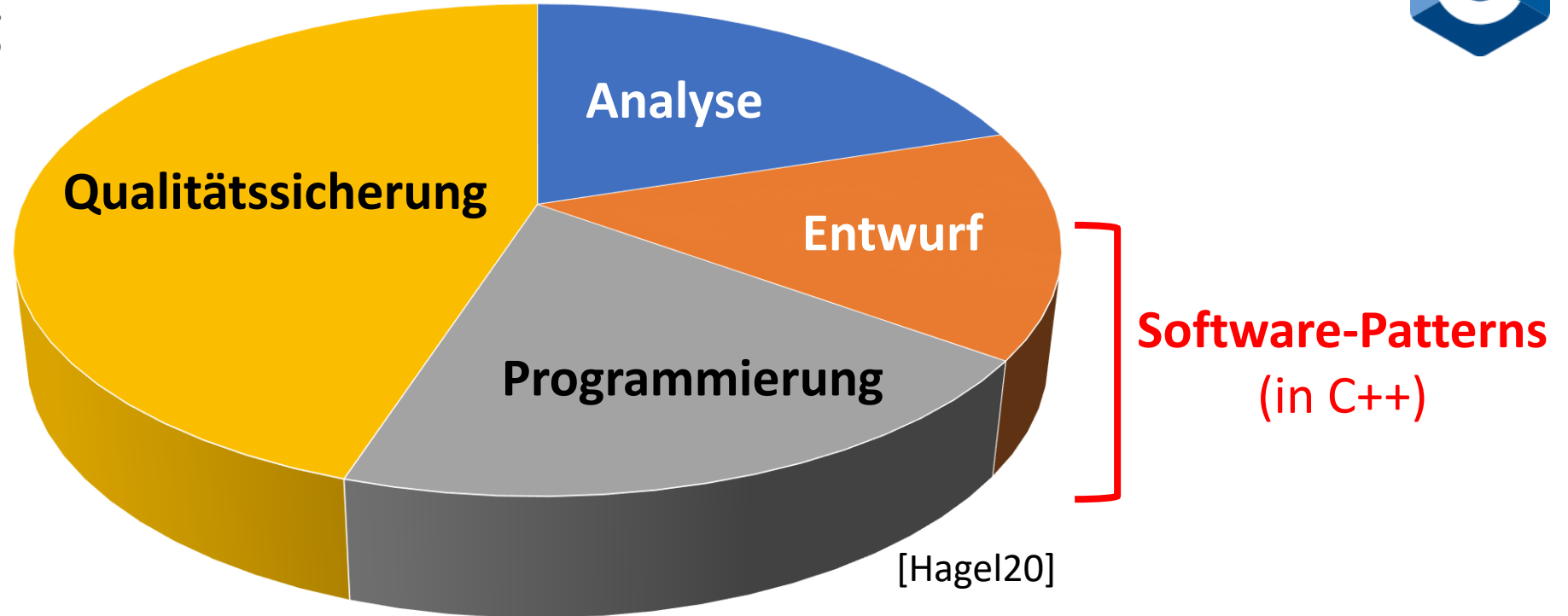
[Clements02]

- Die Softwarearchitektur wird während der **Entwurfsphase** eines Projekts erarbeitet
 - D.h. *vor* der konkreten Implementierung von Programmcode
- Architekturüberlegungen zu einem System ergeben sich aus der **Analyse von Anforderungen** an das System
 - Lastenheft / Pflichtenheft, Funktionale Anforderungen, Technische Anforderungen, Anwendungsfälle, Akteure, ...

[Clements02] : Paul Clements, Documenting Software Architectures: Views and Beyond, 2002

Software-Patterns

Einführung



- Dieses Kapitel befasst sich somit nur mit einem kleinen Teil des Prozesses der Softwareentwicklung
 - Mehr zu den anderen Themen (speziell Analyse, Qualitätssicherung & weitere Muster), siehe LV „**Software Engineering**“

[Hagel20] : Georg Hagel, Software engineering education, 2020



Muster / Patterns

Lernziele

- Die Bedeutung von Mustern (patterns) in der Softwareentwicklung wurde verstanden
- Muster können allgemein und thematisch unterschieden werden
 - Es ist bekannt, mit welcher Art von Mustern sich dieses Vorlesungskapitel beschäftigt

Muster / Patterns

Allgemein



Ein Muster beschreibt den **Kern der Lösung** für ein **wiederkehrendes Problem**, sodass die Lösung auf verschiedene Szenarien angewendet werden kann

[Alexander77]

- Ein Muster kann als **Schablone** einer Lösung gesehen werden, die für ein Problem (angepasst) eingesetzt werden kann
- Muster können in vielen Fällen verhindern, dass das Rad neu erfunden werden muss
- Durch bekannte Muster kann Programmcode schneller von anderen Programmierern verstanden werden

[Alexander77] : Christopher Alexander, A pattern language, 1977

Muster / Patterns

Arten – Allgemein



- Muster werden in der Softwareentwicklung in verschiedene Bereiche kategorisiert
 - **Architekturmuster:** Beschreiben die grundlegende Organisationsstruktur eines Software-Systems (Beispiele: Model-View-Controller (MVC), Schichten, Client-Server, ...)
 - **Analysemuster:** Schablonen für analysierte Klassen und ihre Beziehungen (Beispiele: Transaktion, Juristische Person, Gruppe, ...)
 - **Entwurfsmuster:** Beschreiben die Struktur und Beziehungen eines Subsystems des Programms (Beispiele: Singleton, Factory, Beobachter, Strategie, Zustand, ...)
- Dieses Vorlesungskapitel befasst sich ausschließlich mit **Entwurfsmustern**

Muster / Patterns

Arten – Thematisch



- Entwurfsmuster können weiter thematisch-inhaltlich kategorisiert werden
 - **Erzeugungsmuster:** Beschreiben einen Mechanismus zur Erstellung von Objekten (Beispiele: Singleton, Factory, Builder, ...)
 - **Strukturmuster:** Beschreiben den strukturellen Zusammenhang zwischen Klassen (Beispiele: Decorator, Kompositum, Fassade, ...)
 - **Verhaltensmuster:** Beschreiben die Kommunikation zwischen Klassen (Beispiele: Beobachter, Strategie, Zustand, ...)
- Folgend wird ein Auszug dieser Entwurfsmuster im Detail betrachtet

Muster / Patterns

Implementierung



- Entwurfsmuster sind allgemein – unabhängig von der verwendeten Programmiersprache – definiert
 - Häufig als **UML-Klassendiagramm** dargestellt
- Entwurfsmuster beruhen stark auf Prinzipien der Objektorientierung und Beziehungen zwischen Klassen
 - **Polymorphismus**: Objektdaten (speziell Methoden) werden vererbt und überschrieben
 - **Abhängigkeit & Assoziation**: Objekte stehen in Beziehung zueinander
 - **Aggregation & Komposition**: Objekte setzen sich aus Teilobjekten zusammen
- Alle diese Prinzipien können in C++ realisiert werden
 - Dabei gibt es im Vergleich zu anderen objektorientierten Sprachen jedoch Besonderheiten zu beachten

Muster / Patterns

Zusammenfassung



- **Muster** beschreiben den Kern einer Lösung für ein wiederkehrendes Problem
 - Sie können als **Schablone** für ein konkretes Problem eingesetzt werden
- Muster werden allgemein kategorisiert in **Architektur-, Analyse- und Entwurfsmuster**
- Entwurfsmuster werden thematisch kategorisiert in **Erzeugungs-, Struktur- und Verhaltensmuster**
- Entwurfsmuster sind **objektorientiert** definiert – unabhängig von Programmiersprachen



Singleton

Lernziele

- Die Definition und Funktionsweise des **Singleton** Entwurfsmusters wurde verstanden
- Das Singleton Muster kann in eigenen Projekten umgesetzt werden
 - Hierzu werden Probleme bei der Umsetzung des Musters in C++ und mögliche Lösungen aufgezeigt
 - Abschließend wird eine moderne Implementierung des Musters vorgestellt

Singleton

Muster



- **Wiederkehrendes Problem:** Es darf maximal eine Instanz einer Klasse im gesamten Programm existieren
- **Zweck:** Sicherstellung, dass nur eine Instanz einer Klasse erzeugt bzw. erhalten werden kann
- **Lösung:** Die Klasse hält eine statische Instanz, die nur einmal erstellt und danach zurückgegeben wird
 - Die Klasse erhält eine statische Methode, die die Instanz beim ersten Aufruf erzeugt und bei weiteren Aufrufen zurückgibt
 - Der Konstruktor der Klasse wird private deklariert, sodass Instanzen der Klasse nur innerhalb der Klasse selbst erstellt werden können

Unter „**statisch**“ versteht man programmiersprachenübergreifend die **einmalige Existenz** eines Typs, unabhängig von Instanzen

Singleton

Anwendungsgebiete



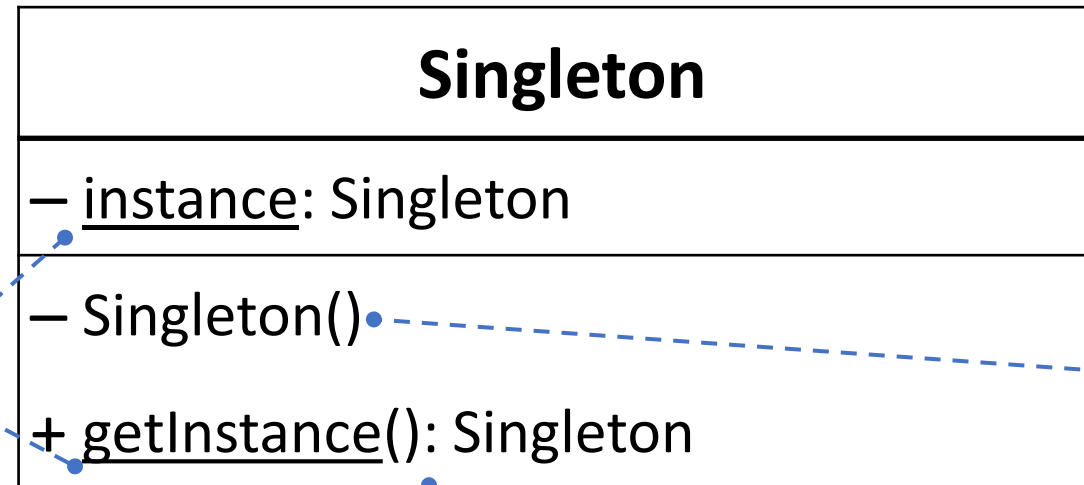
- Singletons werden häufig bei zentralen Objekten einer Anwendung eingesetzt, wie z.B.:
 - **Application:** Klasse, die globale Eigenschaften der Anwendung verwaltet
 - **Configuration:** Zentrale Klasse für die Einstellungen einer Anwendung
 - **Game:** Zentrale Klasse für die Komponenten eines Spiels (häufig in der Spieleentwicklung)
 - ...
 - **Task scheduler:** „Main-Loop“ eines Betriebssystems, Zuteilung von Prozessorzeit und Ausführung von Aufgaben
 - **Nutzende Komponenten:** Einzelne Aufgaben (Tasks) des Betriebssystems
 - **Database:** Laden einer Datenbank, einmalige Existenz der Daten und Manipulationen an der Datenbank
 - **Nutzende Komponenten:** Systeme, die Datensätze der Datenbank bearbeiten
 - **Factory:** Fabrikklassen, die andere Objekte erzeugen, überall erreichbar sein sollen und selbst nur einmal existieren dürfen
 - **Logging:** Einheitliche Ausgabe von Statusmeldungen in eine Log-Datei

Singleton

Muster



- Das Muster kann allgemein durch folgendes UML-Klassendiagramm beschrieben werden:



Statische Member werden in UML durch eine **Unterstreich**ung dargestellt

Ausnahmsweise wird der Konstruktor mit angegeben, um die **private** Sichtbarkeit darzustellen

```
static Singleton getInstance()
{
    if (instance == null) {
        instance = new Singleton();
    }
    return instance;
}
```

Allgemeine Definition
→ Kein C++!

Singleton



Umsetzung – rudimentär

- Nach der allgemeinen Definition kann das Muster wie folgt **rudimentär** in C++ umgesetzt werden:

Singleton
– <u>instance</u> : Singleton
– Singleton()
+ <u>getInstance</u> (): Singleton

```
class Singleton
{
private:
    Singleton() = default;

public:
    static Singleton* getInstance()
    {
        if (m_instance == nullptr)
        {
            m_instance = new Singleton;
        }
        return m_instance;
    }

private:
    static Singleton* m_instance;
};

Singleton* Singleton::m_instance = nullptr;
```

Konstruktor nur zur **private** Deklaration angegeben. Kann vom Compiler generiert werden.

Statische Methode zur einmaligen Erstellung & Erhalt der Instanz

Statische Membervariable der Instanz. Umsetzung nach Definition als Zeiger.

Statische Membervariablen müssen außerhalb der Klassendefinition definiert werden

Singleton



Umsetzung – Problem: Kopien

- Bei dieser Umsetzung ist in C++ jedoch noch nicht sichergestellt, dass nur eine Instanz der Klasse erstellt werden kann
 - Problem: Der Compiler generiert implizit einen **Kopierkonstruktor** für die Klasse

```
class Singleton
{
private:
    Singleton() = default;

public:
    static Singleton* getInstance() { ... }

private:
    static Singleton* m_instance;
};
```

```
int main()
{
    Singleton* instance = Singleton::getInstance();
    Singleton instance2 { *instance }; // 2. Instanz durch implizite Kopie
}
```

Aufruf Kopierkonstruktor



Zweite Instanz des Singletons ist problemlos erstellt worden

Singleton



Umsetzung – Problem: Kopien

- Es muss also zusätzlich der Kopierkonstruktor gelöscht angegeben werden
→ Und nach der **Rule-of-5** dann auch die restlichen Konstruktoren, Zuweisungsoperatoren und der Destruktor angegeben werden

```
class Singleton
{
private:
    Singleton() = default;
public:
    Singleton(const Singleton&) = delete;
    Singleton(Singleton&&) = delete;
    Singleton& operator=(const Singleton&) = delete;
    Singleton& operator=(Singleton&&) = delete;
    ~Singleton() = default;

public:
    static Singleton* getInstance() { ... }

private:
    static Singleton* m_instance;
};
```

```
int main()
{
    Singleton* instance = Singleton::getInstance();
    Singleton instance2 { *instance }; // Compilerfehler
}
```



Zweite Instanz kann nicht mehr
über Kopie erstellt werden

Singleton



Umsetzung – weitere Probleme

- Weiter ist die Implementierung über Zeiger und Funktionen der **manuellen Speicherverwaltung** fehleranfällig (siehe Kapitel „Speicherverwaltung“)
 - Zeiger, Objekt & Allokation kann von anderen Methoden (als „getInstance()“) manipuliert werden
 - Singleton-Klassen folgen nicht dem RAII-Ansatz
 - Wo soll die Instanz letztendlich gelöscht werden?
- Außerdem ist die Implementierung der getInstance()-Methode **nicht Thread-sicher**
 - Mehrere Threads können gleichzeitig auf die Null-Abfrage treffen und dadurch eine zweite Instanz (und einen Memory-Leak) erzeugen
 - Führt in dieser Veranstaltung zu weit...

Singleton

Umsetzung – modern



- In C++ lässt sich die Umsetzung des Musters durch den Einsatz einer **statischen lokalen Variable** deutlich verbessern

```
class Singleton
{
private:
    Singleton() = default;
public:
    Singleton(const Singleton&) = delete;
    Singleton(Singleton&&) = delete;
    Singleton& operator=(const Singleton&) = delete;
    Singleton& operator=(Singleton&&) = delete;
    ~Singleton() = default;
public:
    static Singleton& getInstance()
    {
        static Singleton instance;
        return instance;
    }
private:
    static Singleton* m_instance;
};
```

Rückgabe kann jetzt als Referenz erfolgen

Statische lokale Variable

Statische Membervariable entfällt!

■ Vorteile

- Statische lokale Variable wird automatisch nur beim ersten Aufruf erstellt
- Keine Zeiger – und damit Nullchecks – mehr nötig
- Direkter Zugriff auf Instanz noch weiter eingeschränkt
- Keine statische Membervariable mehr nötig

Singleton



Umsetzung – modern

- Mit dem **C++11** Standard wurde die Umsetzung von Singletons mit statischen lokalen Variablen standardisiert
 - Der **Compiler** muss bei statischen lokalen Variablen seitdem sicherstellen, dass diese Thread-sicher nur einmal erzeugt werden

Moderne Singleton
Implementierung in
C++:

```
class Singleton
{
private:
    Singleton() = default;
public:
    Singleton(const Singleton&) = delete;
    Singleton(Singleton&&) = delete;
    Singleton& operator=(const Singleton&) = delete;
    Singleton& operator=(Singleton&&) = delete;
    ~Singleton() = default;

public:
    static Singleton& getInstance();
};
```

```
Singleton& Singleton::getInstance()
{
    static Singleton instance;
    return instance;
}
```

Seit **C++11** garantiert Thread-sicher

Singleton

Zusammenfassung



- Das Singleton Muster beschreibt die **einmalige Existenz** einer Instanz im gesamten Programmablauf
- Bei der Umsetzung des Singleton Musters gibt es **Probleme** zu beachten:
 - Die **Kopie** der Instanz durch implizite Konstruktoren muss **verhindert** werden
 - Die **manuelle Speicherallokation** der Instanz ist **fehleranfällig**
 - Die Umsetzung ist ohne den Einsatz von Threadlocks **nicht Thread-sicher**
- Seit **C++11** sollte das Muster mit Hilfe einer **statischen lokalen** Variable umgesetzt werden
 - Dadurch wird die korrekte Speicherverwaltung und die Thread-Sicherheit ohne explizite Locks gewährleistet



Factory

Lernziele

- Die Definition und Funktionsweise des **Factory** Musters wurde verstanden
- Das Factory Muster kann in eigenen Projekten umgesetzt werden
 - Hierzu wird die Umsetzung des Musters als einzelne Funktion und als ganze Klasse vorgestellt
 - Außerdem wird ein erweiterter Anwendungsfall für eine abstrakte Fabrik exemplarisch dargestellt

Factory

Muster



- **Wiederkehrendes Problem:** Die Erstellung eines Objekts ist abhängig von Bedingungen, die nicht über einen Konstruktoraufbau allein erfüllt werden können
- **Zweck:** Ausführung weiterer Operationen neben der Allokation und Initialisierung bei der Objekterzeugung
- **Lösung:** Definition einer separaten Funktion oder Klasse, die die Erstellung der Instanzen eines Objekts übernimmt
 - Eine Fabriksfunktion (alleinstehend oder Methode) erstellt die Instanzen einer Klasse
 - Die Funktion berücksichtigt meist weitere Kriterien vor der eigentlichen Erstellung einer Instanz
 - Instanzen werden nicht mehr direkt erstellt – z.B. über den Aufruf des new-Operators – sondern über den Aufruf der Fabriksfunktion

Factory

Anwendungsgebiete



- Fabriken werden häufig für dynamische-austauschbare Typen oder Bedingungen bei der Objekterstellung eingesetzt, wie:
 - **Eingabeverarbeitung:** Der Typ soll entsprechend der Eingabe des Benutzers erzeugt werden
 - **Beispiele:** Nutzer gibt „Hund“ ein → Hund wird erzeugt; Nutzer gibt „Katze“ ein → Katze wird erzeugt, Datei wird abhängig von ihrer Dateiendung geladen, ...
 - **Ressourcenverwaltung:** Es soll nur eine bestimmte Anzahl von Objekten erzeugt werden dürfen
 - **Beispiele:** Nach 1000 Bildern dürfen keine weiteren Bilder mehr geladen werden, um die Speicherplatzauslastung zu kontrollieren
 - **Caching:** Objekte sollen nur einmal geladen und dann für weitere Aufrufe vorgehalten werden
 - **Beispiele:** Laden von Dateien von der Festplatte, Laden einer Textur in den Buffer der Grafikkarte, ...
 - **Konfigurationsabhängigkeit:** Objekte sollen nach dynamischer Konfiguration erstellt werden
 - **Beispiele:** Auf niedrigen Auflösungseinstellungen sollen keine komplexen 3D-Geometrien erstellt werden, sondern 2D-Billboards („Fake Bilder“)

Factory

Fabriksfunktion



- Das Factory Muster kann bereits durch eine einzelne Funktion abgebildet werden = **Fabriksfunktion**
- Beispiel: Fabriksfunktion, die abhängig von Eingabetext verschiedenartige Haustiere erzeugt:

```
Pet* makePet(const std::string& petType)
{
    if (petType == "Dog")
        return new Dog();
    if (petType == "Cat")
        return new Cat();
    if (petType == "Parrot")
        return new Parrot();
    return nullptr;
}
```

Rückgabe: Zeiger auf
abstrakte Basisklasse
→ Polymorphismus!

Erzeugung abhängig
von Eingabeparameter

Fabriksfunktion

```
// Haustier abstrakt
class Pet {...};

// Hund
class Dog : public Pet {...};

// Katze
class Cat : public Pet {...};

// Papagei
class Parrot : public Pet {...};
```

Definitionen Haustiere

Factory

Fabriksfunktion



- Fabriksfunktionen ermöglichen eine **skalierbare Erzeugung** und **polymorphe Nutzung** von Objekten:

```
int main()
{
    // Eingabe Haustiertyp:
    std::string petType;
    std::cin >> petType;

    // Erzeugung konkretes Haustier durch Fabriksfunktion:
    Pet* pet { makePet(petType) };

    // Polymorphe Operation auf Haustier:
    pet->makeNoise();
}
```

Memory-Leaks zu Darstellungszwecken ignoriert

Polymorph:

- „Woof“ für Dog
- „Meow“ für Cat
- „Chirp“ für Parrot

```
class Pet {
public:
    virtual void makeNoise() = 0;
};

class Dog : public Pet {
public:
    void makeNoise() override { std::cout << "Woof"; }
};

class Cat : public Pet {
public:
    void makeNoise() override { std::cout << "Meow"; }
};

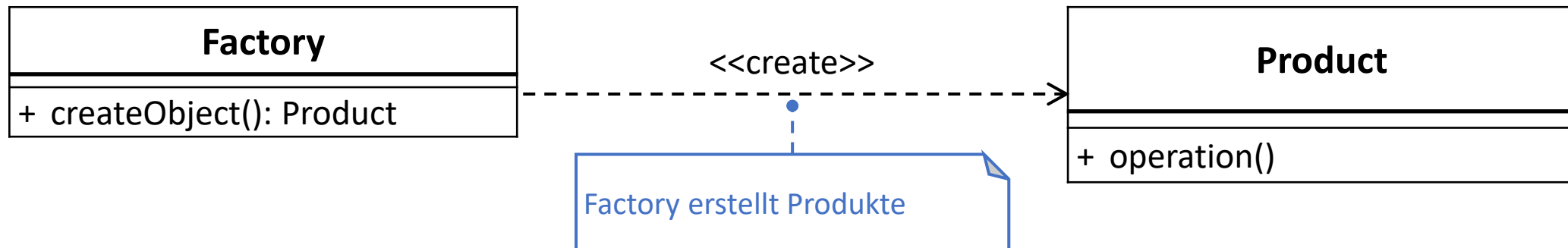
class Parrot : public Pet {
public:
    void makeNoise() override { std::cout << "Chirp"; }
};
```

Factory

Fabriksklasse



- Bei komplexeren Bedingungen zur Objekterzeugung werden **Fabriksklassen** verwendet
 - Fabriksklassen stellen ein oder mehrere **Fabriksfunktionen** zur Verfügung
 - **Membervariablen** können für Bedingungen genutzt werden, die über die Erzeugung eines einzelnen Objekts hinausgehen (z.B. Maximalanzahl erstellter Objekte)
- Eine einfache Fabriksklasse kann durch folgendes UML-Klassendiagramm beschrieben werden:

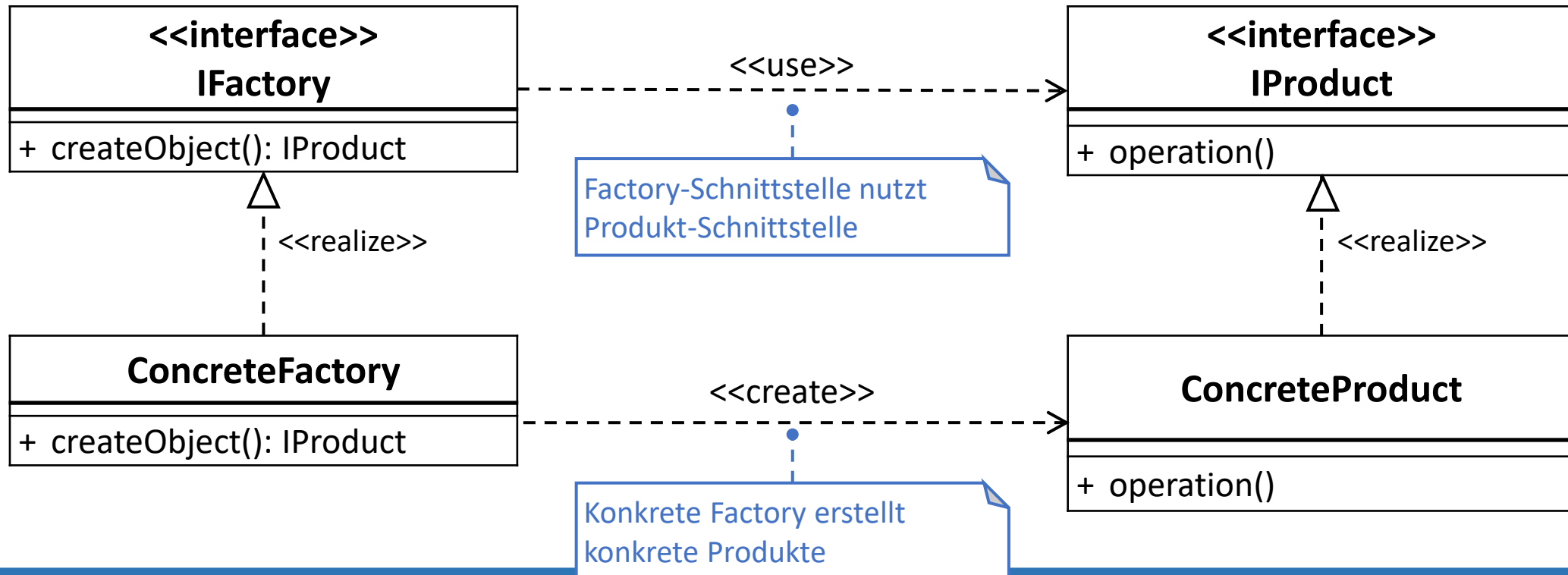


Factory



Abstrakte Fabrikklass

- Für komplexere Programmarchitekturen kann sowohl die Fabrikklass als auch das erzeugte Objekt als Schnittstellenklasse *entkoppelt* werden (= **abstrakte Fabrikklass**):

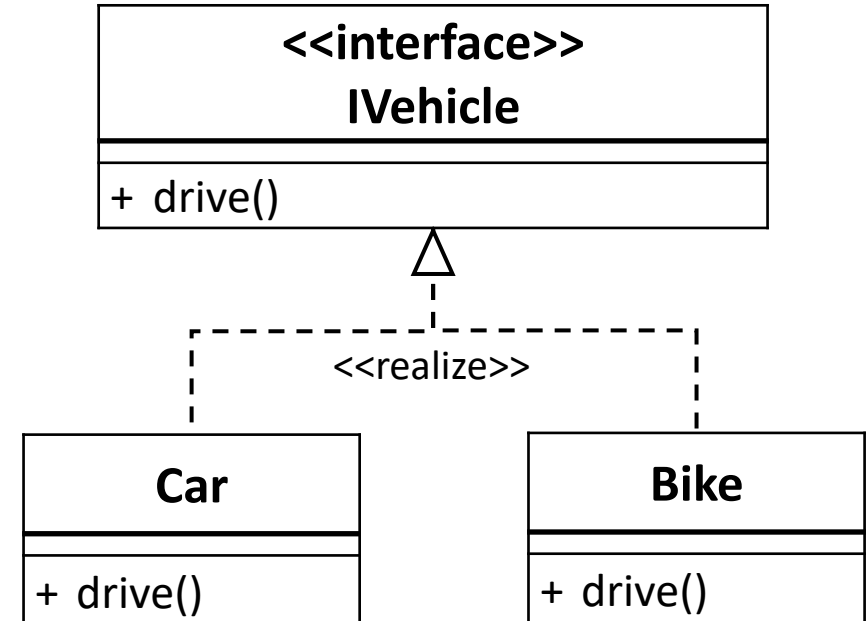
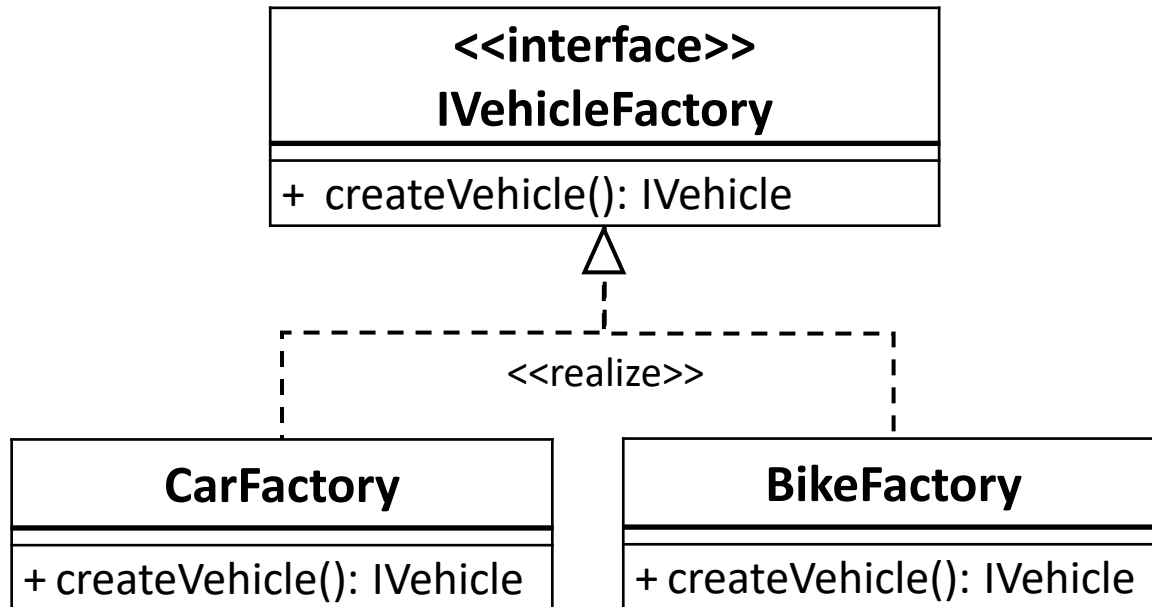


Factory



Abstrakte Fabrikklass – Beispiel

- Durch abstrakte Fabrikklassen können Fabriken und Objekte dynamisch ausgetauscht werden
- Beispiel: Fahrzeugfabrik(en)





Factory

Abstrakte Fabrikklass – Beispiel

■ Umsetzung in C++:

```
class IVehicleFactory
{
public:
    virtual ~IVehicleFactory() = default;
    virtual IVehicle* createVehicle() = 0;
};

class CarFactory : public IVehicleFactory
{
public:
    IVehicle* createVehicle() override { return new Car; }
};

class BikeFactory : public IVehicleFactory
{
public:
    IVehicle* createVehicle() override { return new Bike; }
};
```

Fabrikklassen

```
class IVehicle
{
public:
    virtual ~IVehicle() = default;
    virtual void drive() = 0;
};

class Car : public IVehicle
{
public:
    void drive() override { std::cout << "Driving car"; }
};

class Bike : public IVehicle
{
public:
    void drive() override { std::cout << "Driving bike"; }
};
```

Fahrzeugklassen

Factory



Abstrakte Fabrikklass – Beispiel

- Durch *Entkopplung* können Fabrikklassen dynamisch ausgetauscht werden:

```
void produceAndDriveVehicle(IVehicleFactory& factory)
{
    IVehicle* createdVehicle { factory.createVehicle() };
    createdVehicle->drive();
}
```

Nutzer der Fabrikklassen

```
int main()
{
    CarFactory carFactory;
    produceAndDriveVehicle(carFactory);

    BikeFactory bikeFactory;
    produceAndDriveVehicle(bikeFactory);
}
```

Dynamischer Austausch der Fabriken

A black rectangular box containing the text 'Driving car' on the top line and 'Driving bike' on the bottom line, both in a white, monospaced font.

Memory-Leaks zu Darstellungszwecken ignoriert

Factory



Zusammenfassung

- Das **Factory** Muster beschreibt die isolierte Erstellung von Objektinstanzen einer Klasse
 - Optional durch erweiterte Bedingungen zur Objekterstellung
- Das Factory Muster kann als **alleinstehende Funktion** (= Fabriksfunktion) oder als eigene **Klasse** (= Fabrikklassse) implementiert werden
- Fabrikklassen können über Schnittstellenklassen von konkreten Fabriken **entkoppelt** werden
 - Dies erlaubt den **dynamischen Austausch** von Fabriken zur Laufzeit



Beobachter / Observer

Lernziele

- Die Definition und Funktionsweise des **Beobachter** Musters wurde verstanden
 - Insbesondere wurde der **Kommunikationsprozess** zwischen Klassen des Musters verstanden
- Das Beobachter Muster kann in eigenen Projekten umgesetzt werden
 - Hierzu wird eine vereinheitlichte Implementierung des Musters mittels Template-Klassen vorgestellt

Beobachter / Observer



Muster

- **Wiederkehrendes Problem:** Änderungen an einem Objekt (= Subjekt) haben Auswirkungen auf ein oder mehrere andere Objekte (= Beobachter), die darüber informiert werden müssen
- **Zweck:** Verbreitung der Änderung am Subjekt an die Beobachter
- **Lösung:** Beobachter registrieren sich beim Subjekt und werden für Änderungen des Zustands am Subjekt aufgerufen
 - Das Subjekt erlaubt die Registrierung der Beobachter und gewährleistet den Aufruf einer Methode der Beobachter bei Änderung des Status
 - Beobachter reagieren über eine Schnittstellenfunktion auf die Änderung des Zustands am Subjekt

Beobachter / Observer

Anwendungsgebiete



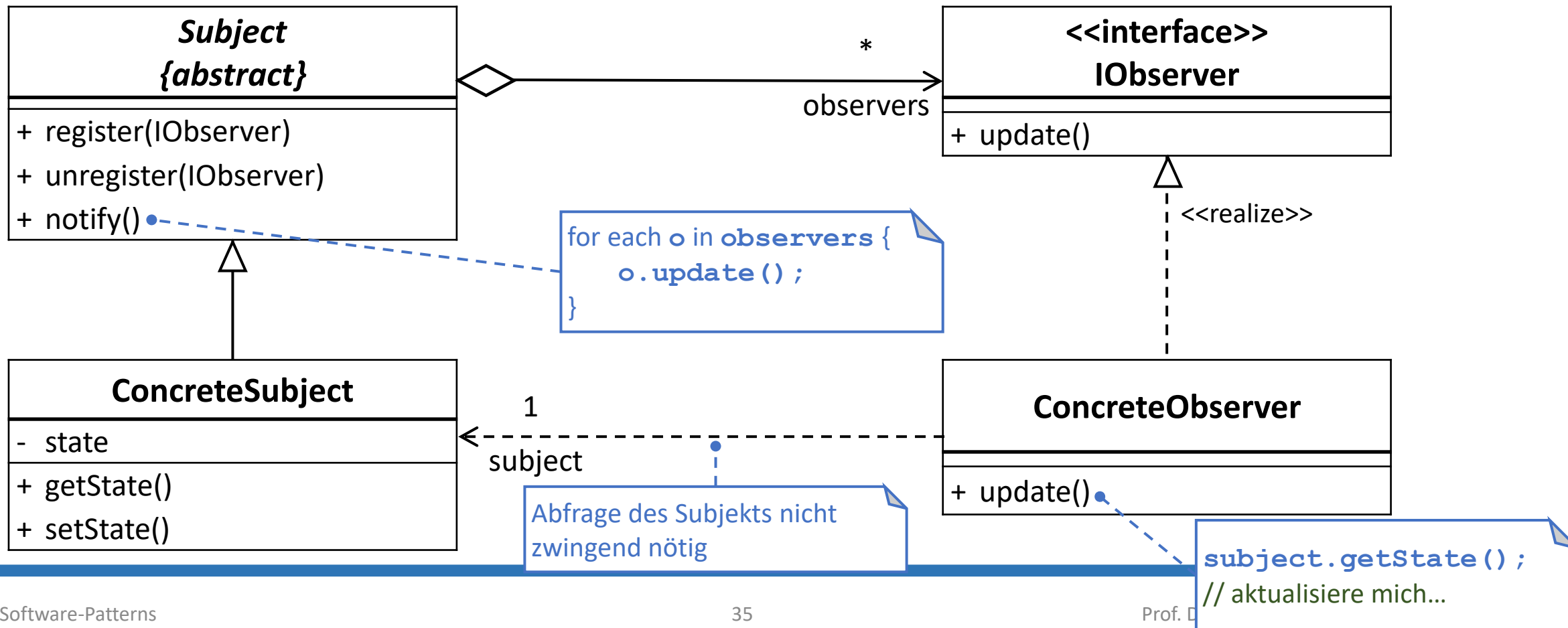
- Das Beobachter Muster wird häufig bei Klassen verwendet, die viel miteinander kommunizieren, wie bei:
 - **Datensynchronisierung:** Die Änderung an einem Objekt wird dem Nutzer visuell dargestellt
 - **Beispiele:** Spieler verliert durch einen Angriff Gesundheitspunkte, aktuelle Temperatur hat sich verändert, der Akkustand des Laptops hat sich verändert, ...
 - **Event-Handling:** Die Reaktion auf ein ausgelöstes Ereignis ist notwendig
 - **Beispiele:** Der Nutzer hat einen Button geklickt, eine neue Zeitschrift wurde veröffentlicht, ein Sensor hat eine Bewegung festgestellt, ...
 - **Logging:** Ein System hat eine Informationsmeldung ausgelöst, die verarbeitet werden muss
 - **Beispiele:** Der Router hat sich mit dem DSL verbunden → Loggen der IP-Adresse, Der Grafiktreiber hat einen Fehler festgestellt → Warnmeldung an den Benutzer, ...

Beobachter / Observer

Muster



- Das Muster kann allgemein durch folgendes UML-Klassendiagramm beschrieben werden:

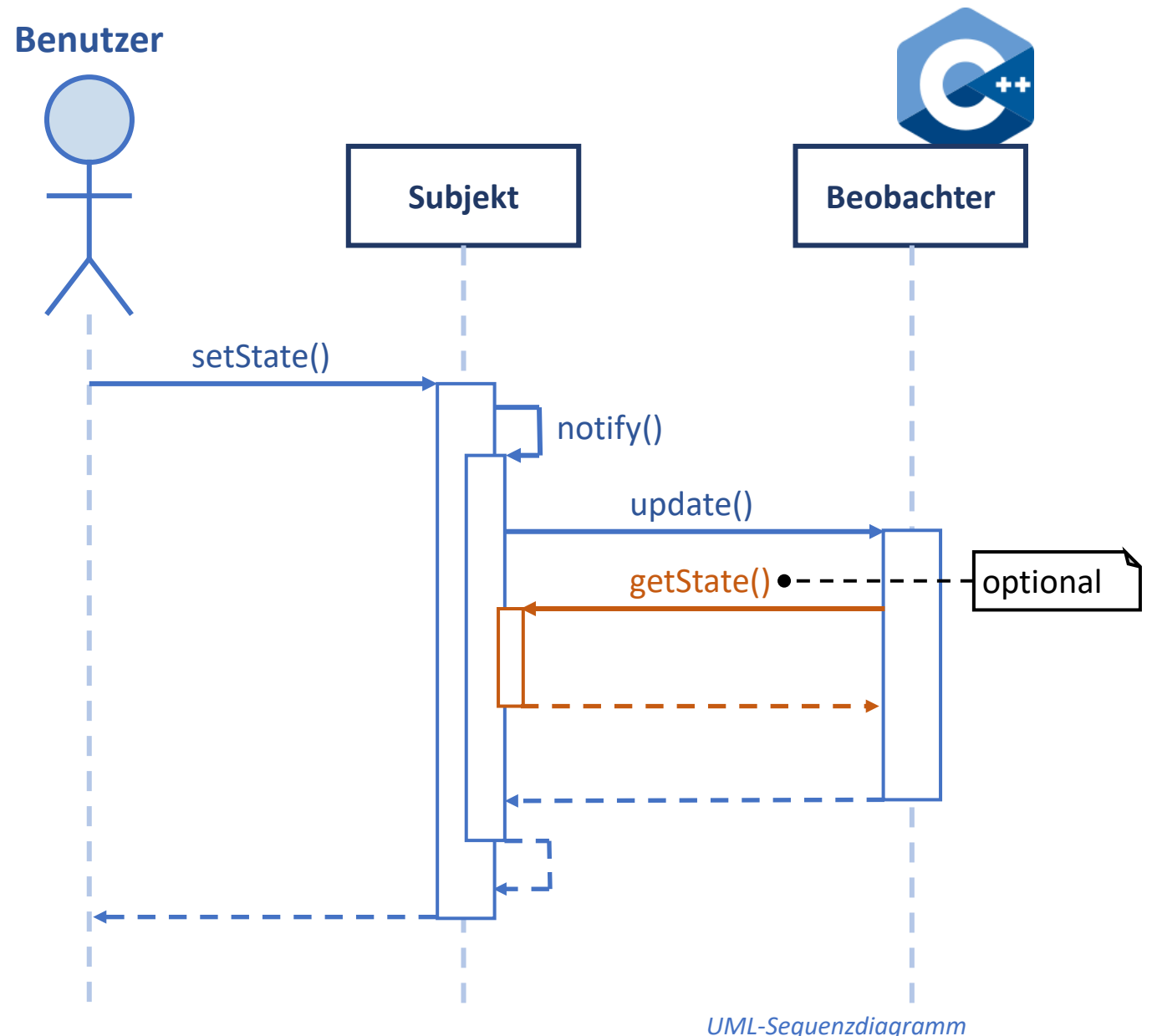


Beobachter / Observer

Ablauf Kommunikation

- Das Beobachter Muster wird häufig für spezielle Anwendungsfälle stark angepasst – der **Ablauf der Kommunikation** bleibt jedoch immer gleich

1. Subjekt: **setState()**
2. Subjekt: **notify()**
3. Beobachter: **update()**
4. *Subjekt: **getState()***
5. Verarbeitung der Änderung im Beobachter



Beobachter / Observer



Beobachter

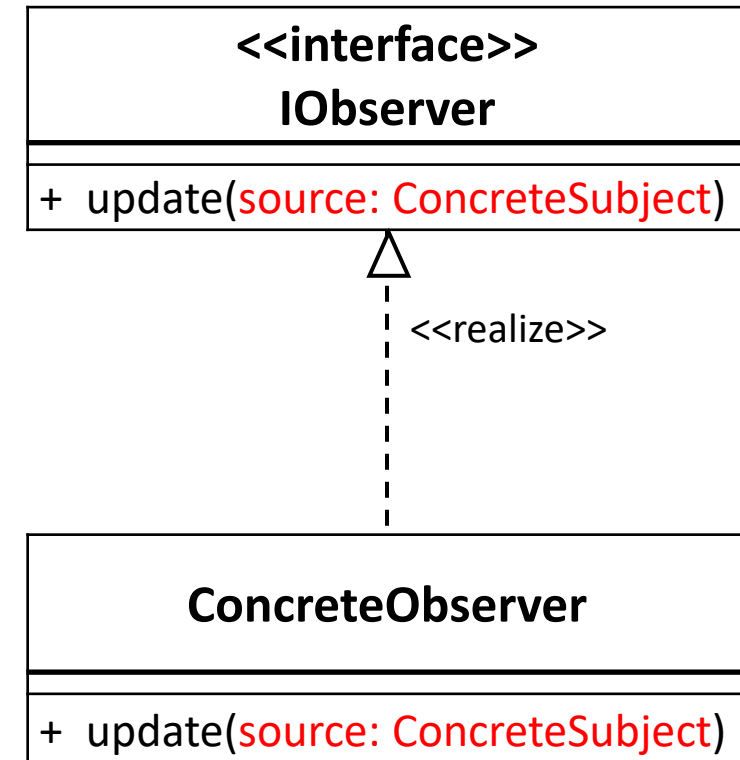
- **Beobachter** Objekte erhalten über eine definierte Schnittstellenmethode Benachrichtigungen über die Änderungen an einem Subjekt
- Die Art der Reaktion auf die Änderung ist dabei stark vom konkreten Anwendungsfall abhängig
 - Häufig wird der neue Zustand des Subjekts abgefragt und weiterverarbeitet
 - Es kann jedoch auch auf die reine Änderung reagiert werden, ohne eine Zustandsänderung zu verarbeiten

Beobachter / Observer

Beobachter – Umsetzung – Hinweis



- Bei der Umsetzung ist nicht vorgegeben, wie (und ob überhaupt) der konkrete Beobachter auf das konkrete Subjekt zugreift
 - Der konkrete Beobachter kann eine **Referenz / Zeiger** auf das konkrete Subjekt als Membervariable halten (*gerichtete Assoziation*)
 - Das Subjekt kann als **Parameter der Update-Methode** weitergereicht werden
 - Das Subjekt kann als **globale Variable / Singleton** zur Verfügung stehen
 - ...
- Der nachfolgende Umsetzungsvorschlag reicht das konkrete Subjekt über die Update-Methode weiter:

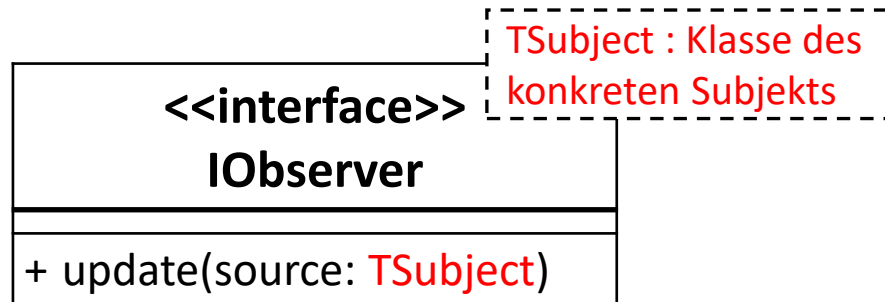


Beobachter / Observer

Beobachter – Umsetzung



- Die **Schnittstellenklasse** „**IObserver**“ kann mit Hilfe von **Templates** einheitlich für alle Subjekte definiert werden:



```
template <typename TSubject>
class IObserver
{
public:
    virtual ~IObserver() = default;
    virtual void update(TSubject& source) = 0;
};
```

Virtuellen Destruktor nicht vergessen!

Observer erhalten Referenz auf konkretes Subjekt. Diese Referenz kann durch *Realisierung* auch **const** deklariert werden!

Beobachter / Observer



Beobachter – Umsetzung – Beispiel

- Konkrete Beobachter realisieren die Schnittstellenklasse unter Angabe des konkreten Subjekts:

Konkreter Beobachter

```
class PersonObserver : public IObserver<Person>
{
public:
    void update(Person& source) override
    {
        std::cout << "Person " << source.getName() << " changed!" << std::endl;
    }
};
```

Konkretes Subjekt, z.B. eine Person
(kann auch `const` realisiert werden)

Realisierungen für
andere konkrete
Subjekte:

```
class CreatureObserver : public IObserver<Creature>
{ ... };
```

```
class OgerObserver : public IObserver<Oger>
{ ... };
```


Beobachter / Observer



Subjekt

- Das **abstrakte Subjekt** Objekt erhält folgende Verantwortungen im Muster:
 - Verwaltung einer **Liste der Beobachter**, die an Statusänderungen interessiert sind
 - Die **An- und Abmeldung** von Beobachtern
 - Die **Benachrichtigung** der Beobachter, wenn sich der Status verändert hat
- Bei der Änderung eines Status – z.B. Setzen einer Membervariable – muss das **konkrete Subjekt** ...
 - **überprüfen**, ob sich der Wert überhaupt **verändert** hat,
 - den neuen Wert der Membervariable **zuweisen** und
 - die Beobachter über die Statusänderung **benachrichtigen**

Beobachter / Observer

Subjekt – Umsetzung



- Die **abstrakte Klasse** „**Subject**“ kann ebenfalls mit Hilfe von **Templates** einheitlich für alle Subjekte definiert werden:

Subject {abstract}	TSubject : Klasse des konkreten Subjekts
- observers: IObserver<TSubject>[*]	
+ register(IObserver<TSubject>)	
+ unregister(IObserver<TSubject>)	
+ notify()	

Bei der Verwendung von Smart-Pointern eignen sich **shared_ptr** aber auch **weak_ptr** oft gut für die Speicherung der Observer-Instanzen

```
template <typename TSubject>
class Subject // auch oft „Observable“ genannt
{
public:
    virtual ~Subject() = default;

    void registerObsrv(IObserver<TSubject>* observer) { ... }

    void unregisterObsrv(IObserver<TSubject>* observer) { ... }

    void notify(TSubject& source) const
    {
        for (IObserver<TSubject>* observer : m_observers)
            observer->update(source);
    }

private:
    std::vector<IObserver<TSubject>*> m_observers;
};
```

Virtuellen Destruktor nicht vergessen!

Container kann auch z.B. **std::set** sein, wenn Observer einzigartig sein sollen

Beobachter / Observer

Subjekt – Umsetzung – Beispiel



- **Konkrete Subjekte** realisieren die abstrakte Basisklasse unter Angabe des konkreten Subjekts – also ihres eigenen Typs! :

Konkretes Subjekt: Person

Person
- age: int
+ getAge(): int
+ setAge(age: int)

```
class Person : public Subject<Person>
{
public:
    Person(const int age) : m_age{ age } {}

    int getAge() const { return m_age; }

    void setAge(const int age)
    {
        if (age == m_age) return;
        m_age = age;
        notify(*this);
    }

private:
    int m_age;
};
```

Template-Realisierung mit
eigenem Typ

1. Überprüfung, ob sich
Zustand überhaupt
verändert hat

2. Änderung des Zustands

3. Benachrichtigung der
Beobachter

Beobachter / Observer

Umsetzung – Verbindung



- Das konkrete Subjekt kann nun mit konkreten Beobachtern verbunden werden:

```
int main()
{
    // Konkretes Subjekt:
    Person person { 20 };

    // Konkrete Beobachter:
    PersonObserver observerA;
    PersonObserver observerB;

    // Registrierung Beobachter beim Subjekt:
    person.registerObsrv(&observerA);
    person.registerObsrv(&observerB);

    // Statusänderungen:
    person.setAge(21);
}
```

Ausgabe:

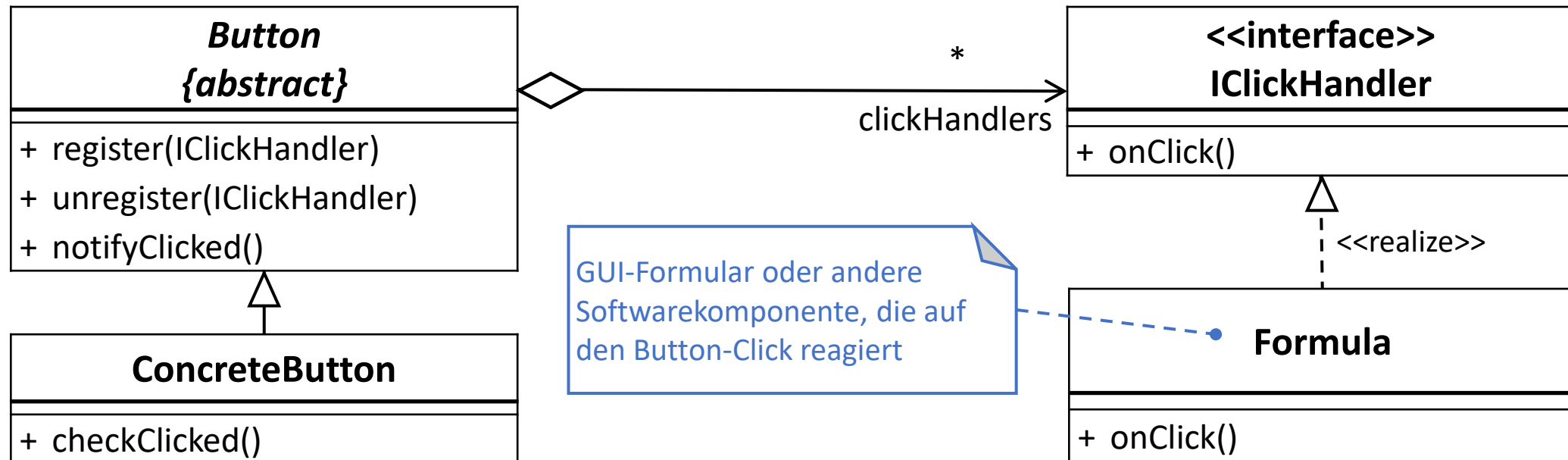
Person changed!
Person changed!

Beobachter / Observer

Event-Handling



- Neben der Veränderung von Zuständen werden Beobachter-Muster auch häufig eingesetzt, um **Events** zu verbreiten und zu behandeln
 - Hierbei ist meist kein Zugriff auf das Subjekt nötig
 - In anderen Sprachen, wie C#, ist diese Art des Beobachter-Musters syntaktisch in die Sprache integriert
- Beispiel: Click-Events eines Buttons



Beobachter / Observer



Zusammenfassung

- Das Beobachter Muster beschreibt die **Verbreitung** einer Statusänderung und die **Reaktion** auf diese
 - **Subjekte**, an denen Änderungen vorgenommen werden, verbreiten die Statusänderung
 - **Beobachter**, die an Änderungen interessiert sind, werden über die Statusänderung benachrichtigt und reagieren auf diese
- Bei der Umsetzung in C++ kann der konkrete Subjekttyp **templatisiert** und abstrakte Subjekte und Beobachter so vereinheitlicht werden
- Das Beobachter Muster kann auch genutzt werden, um **Nachrichten und Events** zu verbreiten
 - Hierbei wird i.d.R. das Subjekt nicht zur Verarbeitung benötigt



- **Welche Vorteile bietet das "Factory" Entwurfsmuster?**
 - a) Es isoliert die Objekterzeugung von anderen Operationen auf dem Objekt.
 - b) Es vereinheitlicht die Objekterzeugung polymorpher Klassenarchitekturen.
 - c) Es verhindert die Erzeugung bestimmter Kindklassen.
 - d) Es erzwingt die Deklaration von virtuellen Methoden.
- **Was ist die Aufgabe des "Singleton" Entwurfsmusters?**
 - a) Sicherstellung, dass Instanzen einer Klasse nur von der Klasse selbst erzeugt werden können.
 - b) Sicherstellung, dass nur eine Instanz einer Klasse erzeugt werden kann.
 - c) Sicherstellung, dass ein Objekt als globale Variable verfügbar ist.
 - d) Sicherstellung, dass der Konstruktor einer Klasse die "private" Sichtbarkeit hat.
- **Bei welchen folgenden Anwendungsgebieten ist der Einsatz des "Beobachter" Musters sinnvoll?**
 - a) Eingabeverarbeitung von Nutzerinteraktionen wie Mausklicks oder Tastendrucke.
 - b) Limitierung der Erstellung von Instanzen einer Klasse auf eine bestimmte Anzahl.
 - c) Realisierung einer Konfigurationsklasse, die nur einmal im gesamten Programm existieren darf.
 - d) Auswahl einer bestimmten Klasse je nach Eingabe des Benutzers.

Software-Patterns

Verständnisfragen

PIN: 150200

<https://poll.hs-kempten.de/poll/>



■ WAHR oder FALSCH?

- Beim Singleton Entwurfsmuster wird der Konstruktor der Klasse "private" definiert.
- Die allgemeine Definition des Singleton Entwurfsmusters kann ohne Zusätze problemlos in C++ umgesetzt werden.
- Das Factory Entwurfsmuster eignet sich besonders gut bei polymorphen Klassenstrukturen.
- Eine Fabriksfunktion hat die alleinige Aufgabe der Objekterzeugung (evtl. abhängig von Bedingungen).
- Eine Fabrikklasse steht mindestens in einer Abhängigkeitsbeziehung zu den Klassen, deren Instanzen sie erstellt.
- Beim Beobachter Entwurfsmuster informiert das Subjekt die Beobachter bei einer Statusänderung.
- Das Subjekt beim Beobachter Entwurfsmuster kann als Schnittstellenklasse umgesetzt werden.